Predicting Chess Game Outcomes from Pre-Game and Early-Game Features: A Comparison of Machine Learning Methods

Hashim Khan
CS-412 Machine Learning
Application Track (solo project)

Project Proposal — October 1, 2026

1 Problem Description

Chess is a two-player game with a rich public record of data, and every game ends in a White win, a Black win, or a draw. This project asks: *how well can machine learning models predict the outcome of an online chess game from information available before or early in the game?*

I will use a dataset of Lichess games containing player ratings, the opening played, the time control, and the move sequence. The main task is classification: predict whether White or Black wins, with draws handled either as a third class or excluded (I will compare both). I will implement and compare several standard algorithms and address three questions:

1. Do ML models beat the obvious baseline of "the higher-rated player wins"?
2. Which features matter most: rating difference, opening choice, time control, or early moves?
3. How does prediction accuracy change as more of the game is revealed (e.g., no moves versus the first 10 moves)?

A key methodological point is **avoiding data leakage**. Features known only after the game ends, such as the termination type (checkmate, resignation, timeout) or the final move count, would inflate accuracy without being meaningful. They will be excluded, and the report will explain why.

2 Brief Survey of Prior Work

Predicting outcomes from player strength has a long history in chess. The Elo rating system [1] and its Bayesian successor Glicko [2] model win probability as a function of rating difference, which motivates the rating-based baseline used here. On the machine learning side, tree ensembles such as Random Forests [3] and gradient boosting (e.g., XGBoost [4]) are strong general-purpose methods for tabular data like this. At the other extreme, deep learning has been used to evaluate and play chess directly, as in DeepChess [5] and AlphaZero [6], but those approaches operate on board positions and need far more data and compute.

Many public Kaggle notebooks train models on the Lichess games dataset [7], but most report a single model's accuracy and do not (a) compare against a rating-based baseline, (b) check for leakage, or (c) study how performance changes as more moves are observed. **This project differs by providing a controlled comparison of multiple algorithms, with a rating-only baseline, explicit leakage control, and an analysis of prediction accuracy versus the amount of game information revealed.**

3 Planned Methodology

- **Data:** Lichess games dataset from Kaggle [7] (about 20,000 games), possibly supplemented with a larger sample from the Lichess open database for the final report.
- **Preprocessing:** remove leaking columns, encode openings and time controls, compute rating difference, and create train/validation/test splits.
- **Models:** rating-difference baseline, Logistic Regression, Decision Tree, k-Nearest Neighbors, Random Forest, and Gradient Boosting, implemented with scikit-learn [8].
- **Evaluation:** accuracy, F1, and confusion matrices with cross-validation; feature importance analysis; and a study of the class imbalance caused by rare draws.
- **Stretch goal:** add features from the first N moves (e.g., $N = 5, 10$) and measure the gain.

4 Preliminary Plan (Milestones)

Date	Milestone
Oct 1	Submit proposal; create GitHub repository with README
Oct 2–10	Download data, exploratory analysis, identify leaking features
Oct 11–20	Implement baseline and first models (Logistic Regression, Decision Tree)
Oct 21–31	Write intermediate report (data description, progress, remaining work)
Nov 1	**Intermediate report due**
Nov 2–12	Add Random Forest, k-NN, Gradient Boosting; tune hyperparameters
Nov 13–20	Early-move feature experiments; feature importance; prepare presentation
Mid Nov onward	Final presentation (5–8 minutes)
Nov 21–30	Write final report (8–10 pages); clean up code and commit history
Dec 1	**Final report due**

All code, the proposal, and the reports will be maintained in GitHub with regular commits.

References

[1] A. E. Elo, *The Rating of Chessplayers, Past and Present*. Arco Publishing, 1978.

[2] M. E. Glickman, "Parameter estimation in large dynamic paired comparison experiments," *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, vol. 48, no. 3, pp. 377–394, 1999.

[3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794, 2016.

[5] O. E. David, N. S. Netanyahu, and L. Wolf, "DeepChess: End-to-End Deep Neural Network for Automatic Learning in Chess," in *Int. Conf. on Artificial Neural Networks (ICANN)*, pp. 88–96, Springer, 2016.

[6] D. Silver et al., "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[7] Kaggle, "Chess Game Dataset (Lichess)," `https://www.kaggle.com/datasets/datasnaek/chess` (data from Lichess.org).

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.